{EPITECH}

CARTRIDGE CUSTOM_

< UN SUJET CUSTOM POUR LES MALADES />

CARTRIDGE CUSTOM

Dans le cadre du projet Cartridge proposé par Epitech, j'ai décidé d'écrire un nouveau sujet parallèle pour les groupes qui le souhaitent afin de plonger plus intensément dans la programmation dite ”bas-niveau” ou encore ”bare-metal”. Ce sujet, qui est un mix entre un gros bootstrap et un sujet, tentera de se positionner comme une porte d'entrée au domaine de la programmation matérielle en couvrant principalement les thèmes suivants :

- ✓ la conception d'outils spécialisés
- ✓ l'utilisation de compilateur spécialisé (cross-compilateur)
- ✓ la conception de driver
- ✓ algorithmie de dessin (dessiner des lignes, des caractères, des cercles, ...)
- ✓ un peu de rétro-ingénierie afin de comprendre le comportement de la machine
- ✓ l'assembleur ARM

Néanmoins, l’une des grosses différences avec le sujet original (au-delà du fond), c’est que l’on va migrer sur la `GameBoy Advance` plutôt que la `GameBoy` ”classique”. La raison principale, c’est qu’on passe d’un processeur z80 à un processeur ARM qui est infiniment plus d’actualité aujourd’hui (contrairement au z80 qui n'existe pratiquement plus (modulo certaines calculatrices de chez `Texas Instrument`)) et j'ai envie de vous faire apprendre quelque chose qui résonne a minima avec le présent.

Ce sujet est divisé en plusieurs ”parties” / ”chapitres” progressifs avec des parutions fréquentes (le sujet ne sera pas disponible d'une traite). Déjà pour me laisser le temps d'écrire, et pour vous, qui tentez de faire ce sujet, le temps de digérer les nouvelles notions qui ne sont pas si simples à appréhender lorsque l'on tombe dessus la première fois. Tout naturellement, je vous invite aussi à me faire des retours pour que je puisse modifier et améliorer le sujet avec le temps.

Parlons maintenant du projet, votre but sera de concevoir un outil de diagnostic matériel. L'idée étant de vous permettre d'observer le comportement de la machine pour comprendre les différents mécanismes internes du matériel (timer, DMA, écran, ...) afin de commencer à l'exploiter à son plein avantage. En parallèle, vous allez concevoir un SDK (une grosse boîte à outils) pour vous aider à déboguer et à naviguer dans le projet et qui vous permettra de vous concentrer sur les notions principales.

Bon courage pour le projet !

Partie I - L'écran rouge

Dans cette première partie de sujet, nous allons principalement nous concentrer sur la mise en place de notre environnement de développement avec comme but final de réussir à afficher un écran rouge dans un émulateur spécialement taillé pour ce sujet.

Je sais que ça fait de la peine dit comme ça, mais vous n'imaginez pas encore toute l'ingénierie qu'il y a derrière cette simple première partie du projet juste pour faire quelque chose d'aussi simple.

Il faut bien garder à l'esprit que, contrairement au programme que vous écrivez sur votre machine et qui tourne grâce à votre système d'exploitation, ici, nous sommes **seuls** avec les composants physiques. Ce qui signifie que si vous voulez afficher un pixel à l'écran... il faut aller discuter directement avec l'écran pour lui dire d'afficher votre pixel... si tant est que l'écran est alimenté, parce que si ce n'est pas le cas, il faut d'abord aller discuter avec la batterie pour leur dire d'aller alimenter l'écran... C'est très schématique, mais on n'est pas loin de la réalité.

Avant de se lancer corps perdu dans la programmation, il est super important de mettre en place, en amont, un environnement de développement et de débogage lorsque l'on se lance dans un projet aussi bas-niveau sur une machine inconnue.

Contrairement, encore une fois, à vos programmes ”classiques”, ici, nous avons beaucoup d'étapes pour pouvoir exécuter notre jeu et la moindre mauvaise instruction peut potentiellement être critique pour le matériel, pouvant l'amener à faire planter physiquement la machine ou pire, la ”bricker” (la casser de manière irrémédiable). Il faut donc bien prendre le temps de mettre en place des outils pour nous aider.

Cette première partie peut donc sembler frustrante à première vue, car nous n'allons pas entrer dans les détails techniques (vous n'allez même rien développer du tout sur la machine, je vous donnerai un bout de code qui fera le travail à votre place), mais elle va nous permettre de poser les fondations solides pour pouvoir naviguer dans votre projet (en plus de vous expliquer les premiers concepts matériels).

todo: image red screen

Contexte historique et sécurité

Trêve de blabla théorique, parlons maintenant de la machine, la fameuse GameBoy Advance (GBA)!

Lorsque l'on désire programmer à très bas niveau (comprenez par là ”lorsque l'on cherche à contrôler le matériel”), il est important de savoir d'où est-ce que l'on part en termes purement d'exécution de programme, et est-ce qu'il y a des obstacles qui nous empêcheraient de prendre le contrôle du matériel ?

Pour notre cas, nous savons trois choses :

1. on est seul à s'exécuter sur le matériel
2. notre code se trouve dans la cartouche de jeu
3. la cartouche semble être exécutée directement au démarrage

Donc, méthodologiquement, la première chose que l'on va essayer de comprendre, c'est la façon dont est structurée une cartouche de la GameBoy Advance.

Cependant... pourquoi une cartouche et pas un autre format ? Une notion super importante à garder à l'esprit lorsque l'on investigue une machine pour la première fois, c'est le contexte économique et historique autour de la console. Du moins, essayer de comprendre ce qui a poussé le constructeur à prendre certaines décisions techniques à ce moment-là de l'histoire.

Dans notre cas, à l'époque de la sortie de la machine (et encore aujourd'hui!), les constructeurs de consoles vendent leurs produits à perte (ou presque)! Le but de l'opération n'est pas de vendre le plus de consoles pour engranger le plus d'argent, mais simplement d'avoir le plus de personnes qui achètent la console. Pourquoi ? Parce que les bénéfices, eux, sont pratiquement exclusivement engendrés par les jeux. Tant par le prix des licences que les studios de développement paient à Nintendo pour pouvoir créer leurs jeux, que le pourcentage du prix taxé sur chaque cartouche vendue.

Le constat est donc clair : que ce soit pour la GBA ou toute autre console, les jeux doivent être suffisamment sécurisés pour que personne ne puisse concevoir / vendre des produits sur une console sans demander directement au constructeur directement (dans notre cas Nintendo).

> C'est d'ailleurs cette position ferme et ce droit de regard invasif sur les studios de développement (car Nintendo a refusé des projets à beaucoup de studios) qui leur ont permis de maintenir une certaine exigence de qualité sur ce qu'ils sortaient ou non sur leur matériel. Cette approche stricte leur a permis de faire remonter la confiance des consommateurs après le crash (américain) du jeu vidéo causé par Atari en 1980 et a permis façonné le monde vidéoludique que l'on connaît aujourd'hui.

Il y a donc une ombre au tableau : la cartouche ne peut pas simplement contenir le code du jeu, sinon tout le monde aurait créé leur propre jeu dessus, il y a donc une arnaque quelque part.

Pour confirmer nos suspicions, nous pouvons simplement prendre une ROM (qui est simplement une copie logicielle (un ”dump”) d'une cartouche GBA) et de simplement essayer d'afficher les instructions pour

tenter de voir si nous avons du code qui ressemble à quelque chose. Si ce n'est pas le cas, c'est qu'il y a effectivement une protection de sécurité mise en place par Nintendo pour éviter de trop facilement concevoir des jeux sur la machine.

Voici l'affichage (fortement simplifié) du début de la cartouche de `Super Mario Advance 2`:

```
ADDR      CODE         INSTRUCTION
   0:     ea00002e     b        0xc0
   4:     51aeff24     ??
   8:     21a29a69     ??
   c:     0a82843d     ??
  10:     ad09e484     ??
  14:     988b2411     ??
  18:     217f81c0     ??
  1c:     19be52a3     ??
  20:     20ce0993     ??
  24:     4a4a4610     ??
  28:     ec3127f8     ??
  2c:     33e8c758     ??
  30:     bfcee382     ??
  34:     94dff485     ??
  ...
  c0:     e3a00012     mov      r0, #18
  c4:     e129f000     msr      CPSR_fc, r0
  c8:     e59fd028     ldr      sp, [pc, #40]   @ 0xf8
  cc:     e3a0001f     mov      r0, #31
  d0:     e129f000     msr      CPSR_fc, r0
  d4:     e59fd018     ldr      sp, [pc, #24]   @ 0xf4
  d8:     e59f1198     ldr      r1, [pc, #408]  @ 0x278
  dc:     e28f0018     add      r0, pc, #24
  e0:     e5810000     str      r0, [r1]
  e4:     e59f1190     ldr      r1, [pc, #400]  @ 0x27c
  ...
```

Du coup, à part la première instruction qui ressemble à quelque chose de valide, tout le reste ne ressemble à rien...Puis, plus tard (vers `0xc0`), on retrouve des instructions qui semblent cohérentes. Il y a donc bien quelque chose de particulier mis en place dans les cartouches, très probablement en lien avec la sécurité de la console. Si on regarde plus en détail la structure d'une cartouche de GBA (qui ont largement été documentées au cours des 25 dernières années), nous pouvons trouver ceci:

todo: schemas format d'une cartouche

Il y a donc bien un en-tête au début de chaque cartouche. Ce qui implique que notre dernière supposition comme quoi ”la cartouche semble être exécutée directement au démarrage” est fausse : il y a bien un programme (le BIOS) qui est exécuté en premier sur la machine, qui va s'occuper de checker que la cartouche contient bien un en-tête particulier.

Notre objectif est donc simple : nous devons comprendre comment est structuré le header afin de le répliquer dans le but de nous faire passer aux yeux de la console comme un jeu légitime. Et c'est justement le premier exercice que nous allons faire.

Pour votre culture générale, je vous recommande chaudement la vidéo Comment un ”docteur” a brisé la sécurité de la Game Boy & GB Advance qui retrace l'histoire du hack de la GameBoy et GameBoy Advance

SDK

Commençons enfin à programmer quelque chose.

Comme expliqué plus haut, nous devons comprendre comment fonctionne le header d'une cartouche pour pouvoir la réplique et nous faire passer pour un jeu légitime aux yeux de la console. Ce qui nous donne une bonne excuse pour concevoir notre premier outil qui devra récupérer le header d'une ROM et y afficher les informations.

Cependant, nous devons prendre en compte le fait que nous allons avoir, à la fin du sujet, plusieurs autres outils à concevoir. Certains d'entre eux reposeront sur d'autres de nos propres outils ou seront carrément interdépendants. Une notion qui revient souvent lorsque l'on commence à avoir beaucoup d'outils est le concept de SDK.

Un `Software Developpement Kit` (SDK), est (généralement) simplement un regroupement d'outils permettant d'avoir, clef en main, un environnement de développement complet pour un domaine ou un type de projet particulier. C'est exactement ce que nous allons tenter de réaliser tout au long de ce sujet.

Bien sûr, nous allons commencer par de simples commandes de visualisation, mais vous allez vite vous rendre compte qu'on va avoir besoin d'un outil pour compiler, puis un autre pour lancer l'émulateur, puis un autre pour convertir des images, ... Il faut penser "environnement de développement", sinon vous allez vous parasiter l'esprit à retenir tout un tas de commandes et d'invocations de scripts qui vous empêcheront de vous concentrer sur le principal à savoir : la programmation matérielle.

Nous allons donc commencer par la conception de l'outil "principal" `cartridge` (ou un autre nom, vous êtes libre là-dessus), qui contiendra une sous-commande `hdr` qui s'occupera d'abstraire la manipulation de l'en-tête de la cartouche.

```
% cartridge --help
Usage: cartridge [OPTIONS] COMMAND [ARGS]...

  Cartridge SDK CLI entry

Options:
  --help  Show this message and exit.

Commands:
  hdr  Cartridge header interface
```

Et pour finir, la sous-sous-commande `dump` qui affichera le contenu d'un en-tête d'une ROM.

```
% cartridge hdr dump --help
Usage: cartridge hdr dump [OPTIONS] [GBA_FILES]...

  dump / display cartridge header information

Options:
  --help  Show this message and exit.
```

Voici un exemple d'affichage d'information de l'en-tête de ma ROM de référence:

```
% cartridge hdr dump super_mario_world_super_2.gba
super_mario_world_super_2.gba:
|-- entry
|    |-- valid: valid
|    |-- raw: 0xea00002e
|    `-- opcode: b 0xc0
|-- nintendo logo:
|    |-- status: valid
|    `-- debugging: disabled
|-- game title: SUPER MARIOB
|-- game code:
|    |-- code: AA2E
|    |-- date: 2001..2003 (old)
|    `-- language: USA/English
|-- marker code:
|    |-- id: 01
|    `-- developer: Nintendo
|-- fixed value: valid (96h)
|-- unit code: 00
|-- device type: 00h
|-- reserved: valid
|-- software_ver: 00h
`-- checksum:
     |-- valid: valid
     |-- rom: 8e
     `-- our: 8e
```

Notez que vous pouvez ajouter autant de commandes que vous voulez. Il n'y a aucune contrainte à ce niveau-là.

Vous êtes libre pour le choix du langage pour la conception du SDK. Cependant, je vous recommande très chaudement le Python (3.10) couplé au framework Click pour la conception de la CLI. Dans tous les cas, utilisez un langage de haut niveau pour la conception d'outil, cela vous fera gagner un temps fou et vous facilitera grandement la vie lorsque nous attaquerons la mise en place du système de compilation ou de conversion d'images.

Bien sûr, je ne vous jette pas sous le bus. Le header a été documenté depuis ces 25 dernières années et vous pouvez en trouver une documentation sur GBATEK qui est très bien (quoiqu'un peu flou sur les bords, ce qui m'arrange pour mon sujet). Je vous recommande fortement de tester sur des ROM ”officielles”, cela vous permettra peut-être de récupérer des informations sensibles dont vous aurez besoin pour contourner la sécurité de la console...

Pas la peine de supporter les ROM ”multiboot” qui contiennent un header plus compliqué. Au vu de nos besoins, vous n'en aurez pas besoin.

Bootstrap

Maintenant que nous avons de quoi récupérer et checker le header du jeu, il est temps de concevoir le deuxième outil qui nous permettra de construire une ROM valide.

Avant ça néanmoins, il nous faut un début de programme qui fait quelque chose, c'est pourquoi je vais vous fournir un bout de code assembleur qui affiche un écran rouge.

```
.text
.global _bootstrap

// Bootstrap code, aggressively request a RED screen
// @notes
//   - do not try this code on real hardware unless you want to destroy your
//       batteries' life...
//   - this code is located right after the CARTRIDGE header
//   - no interruption handler setup...So, aggressively mask all possible
//       interrupt to avoid hardware crash
//   - no stack setup, be careful
_bootstrap:
    // screen configuration
    // @note
    // - select the video mode 3 which is configure a basic VRAM
    // - enable the screen display background 2 (as requested with mode 3)
    // @out
    // - r3 -> new display register configuration
    // - r4 -> old display register configuration
    ldr   r0, =reg_dispcnt_addr
    ldr   r0, [r0]
    ldr   r4, [r0]
    ldr   r2, =reg_dispcnt_config
    ldr   r2, [r2]
    strh  r2, [r0]
    ldr   r3, [r0]

    // VRAM color fill
    // @note
    // - very, very, very slow VRAM update
    // - color the whole VRAM in RED
    // - screen size (w=240,h=160)
    ldr  r0, =vram_addr
    ldr  r0, [r0]
    mov  r2, #31
    mov  r1, #(240*160)
color_loop:
    strh r2, [r0]
    sub  r1, #1
    add  r0, #2
    cmp  r1, #0
    bne  color_loop

    // Waiting loop of death
death_loop:
    b   death_loop
    nop

//---
```


{EPITECH}

```
// Data area
//---

.balign 4
vram_addr:
    .long 0x06000000
reg_dispcnt_addr:
    .long 0x04000000
reg_dispcnt_config:
    .word 0x0403
```

Vous n'avez rien à faire dans cette section, si ce n'est de copier-coller le code au-dessus dans un fichier `bootstrap.S` (oui, un grand `S` et non un petit `s`, nous verrons ça plus tard) que nous compilerons lors de la prochaine étape. Pour l'instant, je vais faire un rapide détour pour vous expliquer un peu plus en détail le fonctionnement de la mémoire de la machine.

todo: schema MMU

Je vais partir du principe que vous êtes assez à l'aise avec les pointeurs et j'espère ne rien vous apprendre si je vous explique qu'un pointeur permet d'accéder à un endroit très précis dans la mémoire. Cependant. Lorsque vous utilisez un pointeur sur votre machine, vous manipulez ce qu'on appelle de la mémoire ”virtuelle” (première partie du schéma). C’est-à-dire que quand vous tentez d'écrire ou de lire via votre pointeur (virtuel), la machine va aller demander au MMU (`Memory Management Unit`), un composant physique qui s'occupe de faire la translation entre mémoire virtuelle et mémoire physique, d'aller lire dans ”la vraie mémoire” (physique).

todo: schemas processus

L'intérêt d'utiliser un MMU de nos jours est en grande partie lié à la sécurité, car le MMU peut mettre des droits sur certains endroits. Cependant, il est aussi surtout utilisé pour faire croire à un processus qu'il est contenu entièrement et de manière continue en mémoire, alors que ce n'est pratiquement jamais le cas. Cela permet aussi d'offrir la possibilité d'exécuter des binaires qui font des tailles gigantesques, dépassant toutes vos mémoires de travail (RAM généralement) présentes sur votre machine en lui donnant l'illusion d'être constamment chargée.

Une stratégie phare avec l'utilisation du MMU, c'est de faire croire à la personne qui demande de la mémoire qu'il a eu ce qu'il voulait en spéculant qu'il ne l'utilise jamais. En revanche, c'est uniquement dès que le demandeur tente de lire, écrire ou exécuter la mémoire que là, le MMU (le kernel pour être plus précis), daigne vraiment allouer de la mémoire.

Typiquement, si vous avez un processeur 32 bits, les adresses (pointeurs) peuvent aller de `0x00000000` à `0xffffffff`, ce qui représente 4 Go. Faites un programme qui boucle simplement sur un `malloc()` qui demande de plus en plus de taille. Vous allez observer qu'il échouera aux alentours de ~4Go. Maintenant faites le même test, mais à chaque fois, tentez simplement de lire ou écrire dans la mémoire. Vous allez observer que le `malloc()` échoue beaucoup plus tôt puisqu'on lui demande ”vraiment” de la mémoire.

todo: schemas memory map

Revenons à notre sujet principal, la GBA. Il n'y a pas de notion de mémoire virtuelle sur cette machine (pour tout un tas de raisons), ce qui signifie que quand vous utilisez un pointeur, vous allez taper directement sur le matériel, il n'y a pas de garde-fou. Donc, imaginez que vous avez un bug dans votre programme qui manipule un porteur qui, par hasard, lorsque vous écrivez dedans, écrase la mémoire de la machine (le BIOS, admettons)... et bien vous cassez définitivement la console.

C'est ce qui est arrivé à `AHelper`, un développeur avant-gardiste des années 2012 sur calculatrice (proche de la GBA), lorsqu'il essayait d'implémenter un bout de code qui chargeait en mémoire des librairies dynamiques en touchant au MMU. L'histoire reste floue, mais il a raté son coup en manipulant des porteurs et la machine a corrompu et/ou effacé le ”boot” de la machine, l'empêchant ainsi de démarrer. La machine ne peut pas être réparée sans intervention matérielle très poussée et a donc été considérée comme ”morte” (ou brickée)

Il faut donc être parfaitement conscient du comportement de la machine à chaque ligne de code que vous écrivez. C'est généralement après ce genre de projet que vous comprenez exactement l'intérêt du langage C et le ”pourquoi” il a été conçu : contrôler des machines (et rien d'autre).

Avec le temps et l'expérience, vous allez vous rendre compte qu'en écrivant du C, modulo certaine sorcellerie, vous pouvez savoir avec une grande précision les instructions assembleur généré...et donc le comportement du matériel à un instant donné.

Je vous rassure tout de même, il est honnêtement peu probable que vous arriviez à casser une machine, il y a énormément de protection matérielle qui empêche de ”bricker” votre machine ou de la réparer après coup.

Repassons un peu sur l'aperçu de la ”map mémoire” de la GameBoy Advance présentée lors du précédent schéma. Vous pouvez voir qu'elle est découpée en ”zones” chacune avec des caractéristiques/ objectifs différents. Par exemple, si on reprend rapidement le code assembleur que je vous ai fourni au début de ce chapitre, vous pouvez voir que la ”variable” `vram_addr` utilise l'addresse `0x06000000` qui, si on en suit le tableau, pointe directement dans la VRAM (`Video RAM`, la mémoire vidéo qui est affichée à constamment l'écran), ce qui semble cohérent pour faire un écran tout rouge.

todo: ecran anti-piratage Game Pak

Une section qui va nous intéresser actuellement dans cette cartographie de la mémoire, c'est la section `Game Pak` qui est le terme technique utilisé par Nintendo pour ”cartouche de jeu”. C'est donc ici que le code de la cartouche sera accessible. Ce qui nous donne, avec toutes les informations glanées depuis le début de ce sujet, le flow de démarrage suivant :

1. allumage de la console, le BIOS démarre automatiquement
2. le BIOS s'occupe d'initialiser la machine (initialiser tous les composants matériels)
3. le BIOS vérifie ensuite que la cartouche respecte la sécurité imposée (le header)
4. le BIOS affiche l'animation de démarrage

5. le BIOS saute à `0x08000000` et donne la main à la cartouche

Nous parlerons plus en détail de pourquoi savoir le fonctionnement exact du flow de démarrage nous est absolument capitale pour commencer à développer plus tard. Néanmoins, vous devriez maintenant comprendre pourquoi le premier champ du header est une simple instruction saut. Cette partie est critique puisque la console l'exécutera quoi qu'il arrive.

Parlons maintenant de la dernière notion à aborder, et qui est aussi le titre de cette section, le bootstrap.

Le bootstrap est littéralement le nom du premier bout de code ”développeur” qui est exécuté juste après le BIOS et qui pourrait s'apparenter à l'étape numéro 6 dans le flow de démarrage que l'on vient de poser.

Son rôle est de mettre une pile (stack) ”utilisateur” (car il n'y en a pas forcément une de mise), initialiser certaines mémoires, initialiser les périphériques matériels pour lever des ambiguïtés de configuration (nous en parlerons lors de la partie deux du sujet) et mettre en place suffisamment de choses pour que du code écrit dans un autre langage puisse être exécuté, ... Pour faire simple, c'est notre BIOS / code de démarrage à nous.

Le bootstrap que je vous ai fourni (le bout de code assembleur) ne fait rien d'autre que mettre l'écran en rouge. Je ne me suis même pas embêté à mettre en place une pile (stack) ou à correctement initialiser la machine. Nous verrons plus tard dans le sujet comment écrire un bootstrap plus élaboré.

Compilation

Bon, j'en dis des choses... c'est bien beau, mais il serait temps de passer à quelque chose de concret : la compilation.

Pour générer un binaire sur une la GameBoy Advance, à votre niveau, il faut simplement savoir que le processeur utilisé est le `ARM7TDMI`. C'est un processeur ARM avec quelques fonctionnalités particulières qui ne nous intéressent pas particulièrement actuellement. Nonobstant, cela nous permet de savoir quel compilateur on peut utiliser, parce que si vous utilisez `gcc` ou `clang` sur votre machine, ils généreront des binaires...pour votre machine et c'est tout.

Il nous faut donc un compilateur "spécial" capable de supporter le processeur de la GBA et nous avons de la chance, puisqu'il en existe un que je vous invite à installer (vous devez installer les deux paquets mentionnés):

- ✓ `arm-none-eabi-gcc`
- ✓ `arm-none-eabi-newlib` (pour avoir accès à certains en-têtes particuliers))

> Pour votre culture générale, que signifient les quatre parties au nom du compilateur ? - `arm` : indique l'architecture visée, ici ARM - `none` : indique le système d'exploitation, ici aucun - `eabi` : indique que nous n'utilisons pas d'interface pour les appels système (`Application Binary Interface`), on tourne directement sur le matériel

Maintenant que nous avons un compilateur, je vous donne tout le flow de compilation de notre mini-application :

```
arm-none-eabi-gcc -mcpu=arm7tdmi -mlittle-endian -nostdlib -T gba.ld -o test.elf bootstrap
    .S
arm-none-eabi-objcopy -O binary test.elf test.bin
cartridge hdr gen -o test.gba -b test.bin
```

Si vous tentez d'effectuer les commandes, ça ne fonctionnera pas ☹ La honte cosmique si vous avez simplement copier-coller les lignes!

On va faire une passe pour décortiquer la première commande :

- ✓ `-mcpu=arm7tdmi` : indique que l'on veut générer du code pour le processeur `ARM7TDMI`
- ✓ `-mlittle-endian` : indique que l'on veut générer du code en little-endian (pas obligatoire, car mis automatiquement + ignoré par GCC)
- ✓ `-nostdlib` : indique que l'on ne veut pas de librairie standard (même si on a newlib)
- ✓ `-T gba.ld` : indique que l'on veut utiliser le inker script `gba.ld`... que vous n'avez pas encore

Ensuite, concernant les deux autres commandes, le `objcopy` prend le fichier au format `ELF` qui est utile pour

les systèmes d'exploitation, mais parfaitement inutile pour nous qui n'en avons pas, et on y extrait uniquement le code binaire en brut.

Et pour finir, on prend le binaire et on y ajoute notre header pour en faire un vrai ROM. Vous aurez compris que cette commande sera à implémenter à la fin de cette première partie.

todo: schemas flow de compilation

En parlant de linker script, je vais tenter de vous expliquer très rapidement son intérêt et pourquoi on en utilise un.

Le rôle d'un linker script, c'est d'indiquer au "linker" (la toute dernière étape lors de la génération d'un programme), comment "structurer" notre binaire. Quelles fonctions vont se trouver à quelle adresse en mémoire dans le but de respecter la cartographie des mémoires de la machine (la memory map). Je suis conscient que mon explication reste très floue, mais nous aurons l'occasion d'en parler beaucoup plus en profondeur lors de la partie 3 du sujet.

À savoir que vous utilisez forcément un linker script sans le savoir, car il est caché et ajouté automatiquement lorsque vous appelez `gcc` sur votre machine (pareil pour `-lc` de la librairie C qui est ajouté dans votre dos à chaque fois (d'où le `-nostdlib`)). Vous pouvez faire apparaître le linker-script de votre machine avec `gcc -Wl,--verbose`.

```
/* custom linker script in order to generate a GameBoyAdvance binary */
OUTPUT_FORMAT("elf32-littlearm", "elf32-bigarm", "elf32-littlearm")
OUTPUT_ARCH(arm)
ENTRY(_bootstrap)

/* address only the cartridge memory area
 * @notes:
 *   - skip the cartridge header
 *   - other memory area will be cover soon.. */
MEMORY {
    rom  (rx) : ORIGIN = 0x080000c0, LENGTH = 32M
}

/* aggressively keep code binary and force it to be located at the start of the ROM */
SECTIONS {
    .text : {
        KEEP(*(.text));
    } > rom
}
```

Voici le fameux `gba.ld`. Comme vous pouvez le voir, il est extrêmement minimaliste comparé à celui de votre machine et dit basiquement au compilateur : ne fait rien d'autre que de mettre tout le code assembleur à l'adresse `0x080000c0`... qui se trouve être, si on regarde notre tableau encore une fois, juste après la section du `Game Pak`. Vous devriez commencer à comprendre pourquoi on ne commence pas à `0x08000000` directement, mais avec un décalage de `0xc0` octets.

Avec cette dernière pièce du puzzle, vous devez avoir suffisamment d'informations pour au moins réussir à générer le binaire brut du projet. D'ailleurs, vous pouvez vérifier par vous-même en utilisant `arm-none-eabi-objdump -m arm7tdmi -b binary -D test.bin` pour vérifier que les instructions sont bien celles qui sont écrites dans le `bootstrap.S`. Nous aurons l'occasion de reparler de la notion de ”debug” plus tard.

Par contre...vous avez la même impression que moi ? Ce n'est pas complètement chiant de tout écrire à la main ? Pourquoi ne pas ajouter une entrée dans notre superbe SDK ? Ajouter la sous-commande `cartridge build` qui buildera automatiquement votre projet. (Je conçois que là, ce n'est pas forcément intéressant d'avoir une commande qui va juste lancer votre Makefile à votre place...mais, pour la suite, vous verrez un intérêt (et encore plus avec le temps)).

Faites un simple enrobage autour de Make pour l'instant et ignorez la génération du header que nous n'avons pas encore implémentée. D'ailleurs, en parlant de ça...

Cartridge

Maintenant que nous avons un binaire pur, il ne manque plus qu'à générer la ROM finale. En soi, nous savons déjà comment fonctionne le header...reste plus qu'à faire le processus de génération.

Implementer la commande suivante:

```
% cartridge hdr gen --help
Usage: cartridge hdr gen [OPTIONS]

  header generation

Options:
  -b, --binary FILE  input binary file  [required]
  -o, --output FILE  ouput filename  [required]
  -n, --name TEXT    internal cartridge name (default: filename)
  --help             Show this message and exit.
```

Vous pouvez ajouter d'autres flags pour le fun si vous voulez. Je vous recommande aussi d'utiliser le `cartridge hdr dump` pour vous assurer que votre génération est bonne.

Émulation

Maintenant reste la partie ”fun” du projet : l'émulation.

Comme on vise à faire du bas niveau ainsi qu'un ”observateur” de matériel, il nous faut un émulateur qui est orienté pour faire de l'observation matérielle. Un des émulateurs phares pour faire du développement du GBA s'appelle `no$gba`, malheureusement, il n'est pas disponible sur Linux et son interface reste extrêmement rudimentaire.

C'est pourquoi j'ai trouvé l'émulateur `ayyboy-advance` qui reste très pertinent dans son design pour faire ce que l'on souhaite faire. Cependant, l'outil est ”mal conçu” sur certains points et fonctionne uniquement sur de très grands écrans (la taille de la fenêtre est totalement fixé a la main, ...). C'est pourquoi, j'ai pris sur moi de forker de dépôt et d'y ajouter des fonctionnalités et des correctifs spécifiques à notre sujet. Je vous invite donc fortement à utiliser ma variante du projet (et à y participer si le coeur vous en dit).

todo: screenshot

Parmi les micro-fonctionnalités que j'ai ajoutées (en plus de quelques fix), vous avez un flag `--debuggeur` qui permet directement d'entrer en mode ”débogueur”, ce qui devrait grandement vous être utile par la suite.

> Le fork est encore expérimental, je n'ai pas encore eu le temps de corriger certains bugs de GUI pour qu'il puisse être complètement utilisable dans son ensemble. Néanmoins, il reste pratique pour notre sujet et je continuerai à l'améliorer avec le temps. Je vous tiendrai au courant des avancées.

Votre objectif est donc de réussir à build le projet et à lancer votre jeu sur l'émulateur. Théoriquement, vous ne devriez pas croiser de problème particulier. Il vous restera plus qu'à ajouter une commande dans votre SDK pour lancer automatiquement l'émulateur à la fin de chaque compilation.

> Vous pouvez simplement ajouter une règle dans votre Makefile (eg `make emu`), mais vous pouvez aussi créer une nouvelle sous-commande qui s'occupera de récupérer l'émulateur, le build et le lancer. C'est comme vous le sentez pour l'instant.

Voilà! Nous avons maintenant de bonnes bases pour entrer un peu plus en profondeur sur la programmation bas-niveau!

À suivre...

v 0.1.0-rc0

{EPITECH}